FECAP – Fundação Escola de Comércio Álvares Penteado
Análise e Desenvolvimento de Sistemas · Projeto Interdisciplinar

Desenvolvimento Web Full Stack

Segunda entrega – evidências técnicas

KFKA – Plataforma de Acompanhamento Escolar

Integrantes
João Pedro Teixeira dos Santos
Daniel Bandini Grande
Raphael Carmo Mendes
Guilherme Koide Rangel

São Paulo, novembro de 2026

Identificação do projeto

O sistema desenvolvido pelo grupo MarketMob chama-se **KFKA – Plataforma de Acompanhamento Escolar**. Ele serve para registrar, revisar, publicar e acompanhar o desempenho bimestral de alunos do Ensino Fundamental, ligando a escola, os professores e os responsáveis. O grupo é formado por João Pedro Teixeira dos Santos, Daniel Bandini Grande, Raphael Carmo Mendes e Guilherme Koide Rangel.

A API foi feita em Node.js 24 com Express 5, o banco de dados é o MySQL 8 e a interface foi construída em React 19 com Vite. A segurança usa JWT e bcrypt, e a hospedagem fica no Railway. O código-fonte está no repositório **https://github.com/2026-2-NADS2/Projeto12**, e o sistema publicado pode ser acessado em **https://projeto12-production.up.railway.app**.

Acessos de demonstração

O ambiente publicado usa dados fictícios (24 alunos, 3 turmas, 5 professores e 12 famílias), criados pelo script de carga do próprio projeto. Para entrar como **administrador**, use o e-mail admin@escola.com com a senha Admin2026. Para o perfil de **professor**, use beatriz@escola.com ou carlos@escola.com, ambos com a senha Prof2026. Para o perfil de **responsável**, use maria@email.com (mãe de Lucas e Júlia) com a senha Resp2026.

Como ler este documento

Cada uma das sete seções a seguir corresponde a um item pedido na entrega. Em cada uma há uma explicação do que foi feito, onde isso está no código e exemplos reais de requisição e resposta da API.

Identificação do projeto 2
Acessos de demonstração....2
Como ler este documento....2
1. API com as principais rotas....4
1.1 Rotas principais....4
1.2 Exemplo de chamada....5
2. Persistência em banco de dados....6
2.1 Principais tabelas....6
2.2 Como o código acessa o banco....7
3. Autenticação (cadastro e login) e rota protegida....8
3.1 Cadastro....8
3.2 Login....8
3.3 Rotas protegidas....9
4. Validação e erros padronizados....12
4.1 Códigos usados....12
4.2 Validação também na interface....13
5. Segredos fora do código....16
6. Aplicação publicada (front e back em produção)....19
6.1 Como o deploy funciona....19
7. Repositório com commits de todos os integrantes....22
Resumo do atendimento....24

1. API com as principais rotas

A API é um servidor **Node.js com Express 5** organizado em camadas, cada uma com uma responsabilidade:

- **Rotas (routes/):** recebem a requisição HTTP, validam o formato dos dados e devolvem a resposta.
- **Serviços (services/):** aplicam as regras de negócio, como o período do bimestre, o fluxo de revisão e a força da senha.
- **Repositórios (repositories/):** executam o SQL no banco.
- **Database (core/Database.js):** mantém o pool de conexões com o MySQL.

Todas as respostas são em JSON. As listas grandes (alunos, professores e responsáveis) são paginadas no formato { itens, pagina, porPagina, total, totalPaginas } e aceitam busca por texto (?busca=).

1.1 Rotas principais

As rotas de **autenticação** são o login (POST /auth/login), a consulta do usuário logado (GET /auth/me), a recuperação de senha (POST /auth/recuperar-senha e POST /auth/redefinir-senha) e a troca de senha (POST /auth/alterar-senha). O login e a recuperação são públicos; as demais exigem estar logado.

Os **cadastros**, de uso exclusivo do administrador, seguem o mesmo padrão em /alunos, /professores, /responsaveis, /areas, /disciplinas, /turmas e /tags: GET lista, POST cria, PUT /:id altera e DELETE /:id inativa o registro. Os **vínculos** também são do administrador: /turmas/:id/alunos matricula alunos, /turmas/:id/disciplinas aloca disciplinas e professores, e /responsaveis/:id/alunos liga responsáveis aos alunos.

Os **bimestres** são consultados em GET /bimestres, aberto também a professores e responsáveis, e configurados pelo administrador em PUT /bimestres/:ano/:numero.

O **professor** vê suas turmas em GET /professor/turmas, lê e grava cada lançamento em /professor/turmas/:t/disciplinas/:d/bimestres/:n/alunos/:a (GET e PUT), envia para revisão em POST /professor/acompanhamentos/:id/enviar e acompanha os retornos das famílias em GET /professor/retornos.

Na **revisão**, o administrador lista os acompanhamentos em GET /revisao/acompanhamentos, abre um deles com o histórico em GET /revisao/acompanhamentos/:id, altera com PUT e decide entre POST /:id/devolver e POST /:id/publicar.

O **responsável** vê os alunos vinculados em GET /responsavel/alunos, o relatório em GET /responsavel/alunos/:id/relatorio, baixa o PDF em /relatorio/pdf e registra a ciência em POST /:id/ciencia. Por fim, os **relatórios gerenciais** ficam em

/relatorios/medias, /status, /tags e /retornos, e a exportação em /relatorios/exportar?formato=xlsx ou csv.

1.2 Exemplo de chamada

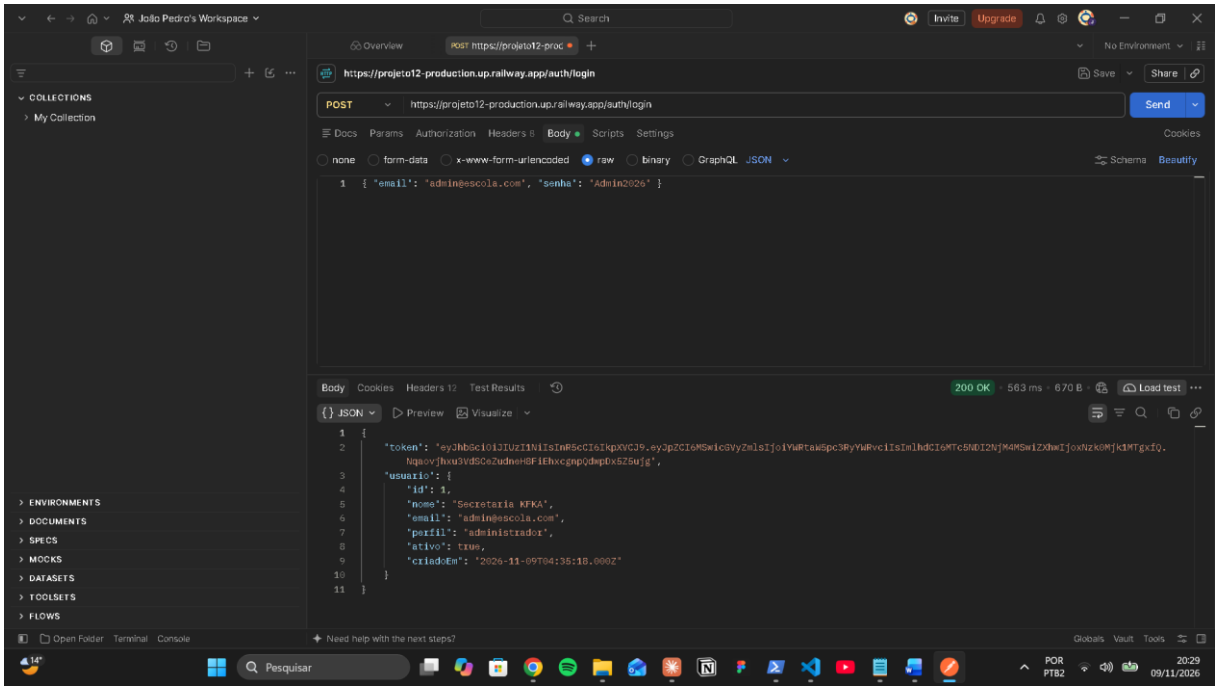
Listagem paginada de alunos com busca (requisição autenticada):

```
GET /alunos?pagina=1&porPagina=2&busca=Lucas
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

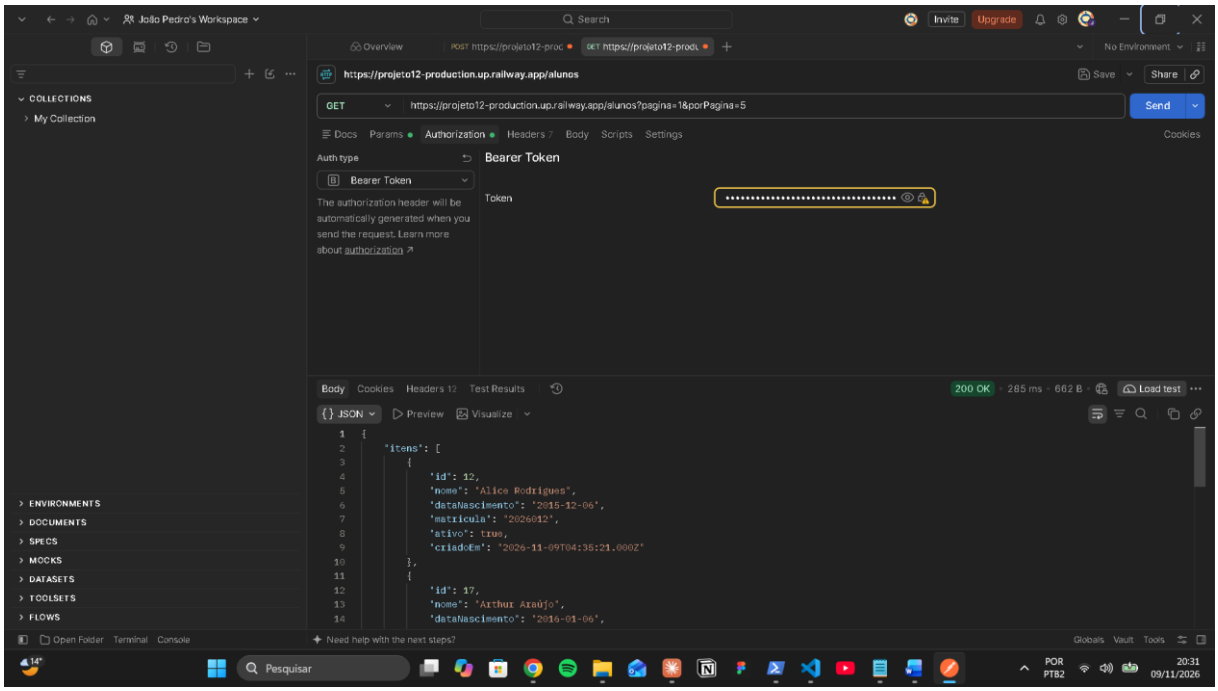
200 OK
{ "itens": [ { "id": 1, "nome": "Lucas Oliveira", "matricula": "2026001",
               "dataNascimento": "2016-03-14" } ],
  "pagina": 1, "porPagina": 2, "total": 1, "totalPaginas": 1 }
```

PRINT: Postman – GET /alunos retornando 200 com a lista paginada
(aba Authorization com Bearer Token preenchido; mostrar o status 200 e o JSON)

Figura 1 – Listagem de alunos pela API



PRINT: Postman – GET /revisao/acompanhamentos/:id mostrando o acompanhamento e o histórico]



2. Persistência em banco de dados

Os dados ficam em um banco **MySQL 8**. O script backend/database/schema.sql cria todas as tabelas, e o comando npm run dados carrega os dados fictícios de demonstração. Em produção, o banco roda no Railway, em um serviço MySQL separado do servidor da aplicação.

2.1 Principais tabelas

A tabela **usuarios** guarda as contas de acesso: nome, e-mail único, senha em hash, perfil e situação. As tabelas **alunos**, **professores** e **responsaveis** guardam os cadastros de pessoas; professor e responsável são ligados um a um a um usuário. A estrutura escolar fica em **areas_disciplina**, **disciplinas**, **turmas** e **tags**.

Os vínculos de muitos para muitos ficam em **turma_aluno**, **turma_disciplina_professor** e **responsavel_aluno**, com chaves únicas que impedem duplicidade. A tabela **bimestres** registra a abertura e o encerramento da digitação de cada bimestre.

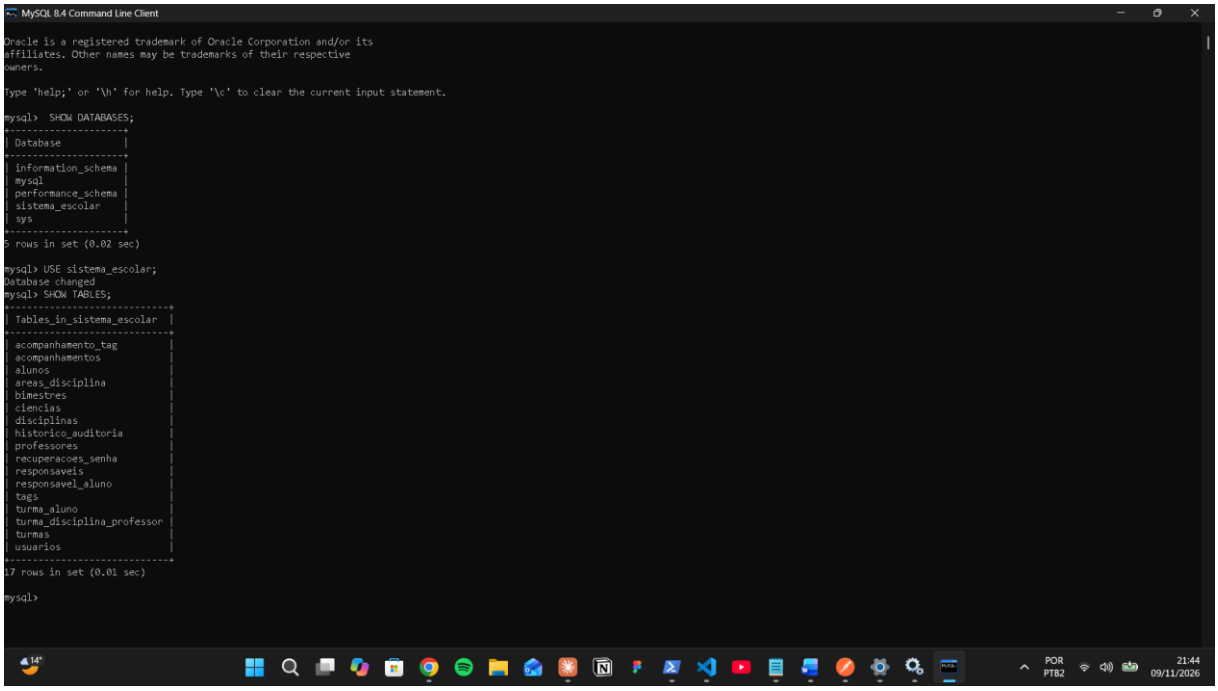
Os lançamentos ficam em **acompanhamentos** (descrição, média e estado editorial de cada aluno, disciplina e bimestre) e as tags escolhidas em **acompanhamento_tag**. A auditoria fica em **historico_auditoria**, com usuário, data e hora, estado anterior e posterior. Há ainda **ciencias**, com a ciência e o retorno do responsável, e **recuperacoes_senha**, com os links temporários de recuperação de senha, guardados em hash.

2.2 Como o código acessa o banco

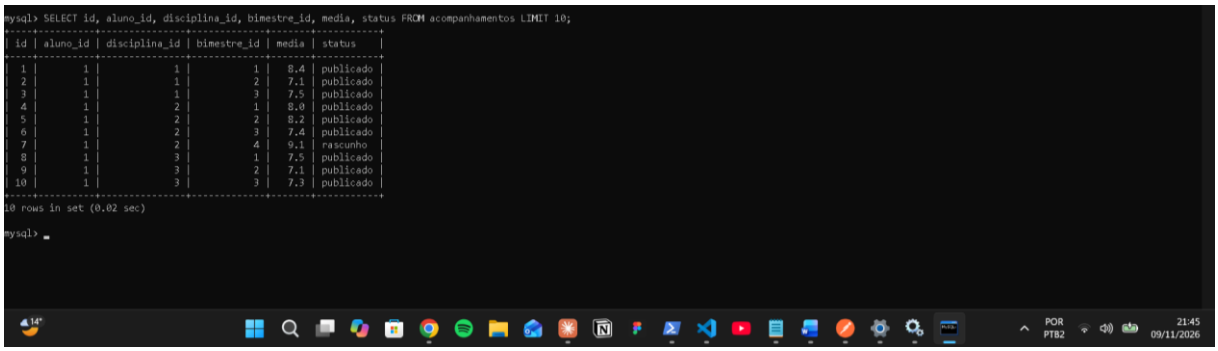
Os repositórios usam **consultas parametrizadas** do mysql2 (? no lugar dos valores). O valor nunca é colado no texto do SQL, o que impede injeção de SQL. Por exemplo, uma busca com ' OR 1=1 -- devolve 200 com a lista vazia, e não todos os registros.

Operações que alteram várias tabelas, como criar um professor junto com a conta de login dele, usam **transações**. Se uma parte falha, nada é gravado pela metade.

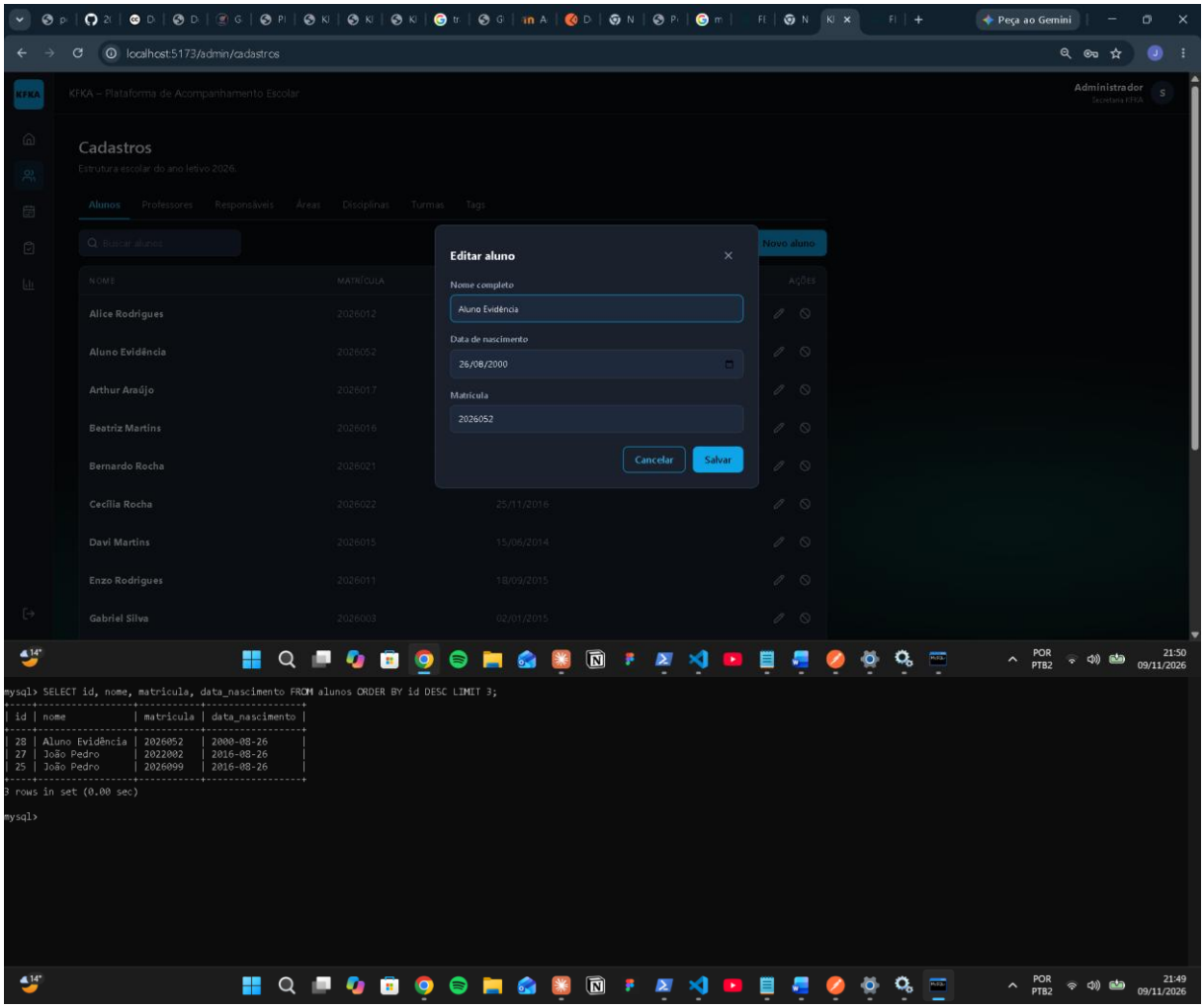
PRINT: MySQL Workbench – lista das tabelas do banco sistema_escolar
(painel Schemas expandido em Tables)



PRINT: MySQL Workbench – SELECT em acompanhamentos mostrando os dados gravados
(ex.: SELECT id, aluno_id, disciplina_id, bimestre_id, media, status FROM acompanhamentos LIMIT 10;)



PRINT: Antes e depois: cadastrar um aluno pelo site e mostrar a nova linha no SELECT



3. Autenticação (cadastro e login) e rota protegida

3.1 Cadastro

Por ser um sistema escolar, **não existe autocadastro público**: quem cria as contas de professores e responsáveis é o administrador, na tela Cadastros (POST /professores e POST /responsaveis). Essa decisão impede que qualquer pessoa crie uma conta e veja dados de alunos.

- A senha é validada (8 a 72 caracteres, com letra e número) e o e-mail precisa ter formato válido.
- A senha é guardada como **hash bcrypt** (custo 10), nunca em texto puro. No banco aparece algo como $2b$10$....
- E-mail repetido é recusado com o código 409.

3.2 Login

POST /auth/login compara a senha com o hash (bcrypt.compare) e, se estiver correta, devolve um **token JWT** válido por 8 horas, com o id e o perfil do usuário.

```
POST /auth/login
{ "email": "admin@escola.com", "senha": "Admin2026" }

200 OK
{ "token": "eyJhbGciOiJIUzI1NiIs...",
  "usuario": { "id": 1, "nome": "Secretaria KFKA", "email": "admin@escola.com",
               "perfil": "administrador", "ativo": true } }
```

Se o e-mail ou a senha estiverem errados, a mensagem é a mesma nos dois casos ("E-mail ou senha inválidos."), de propósito, para não revelar quais e-mails existem no sistema.

3.3 Rotas protegidas

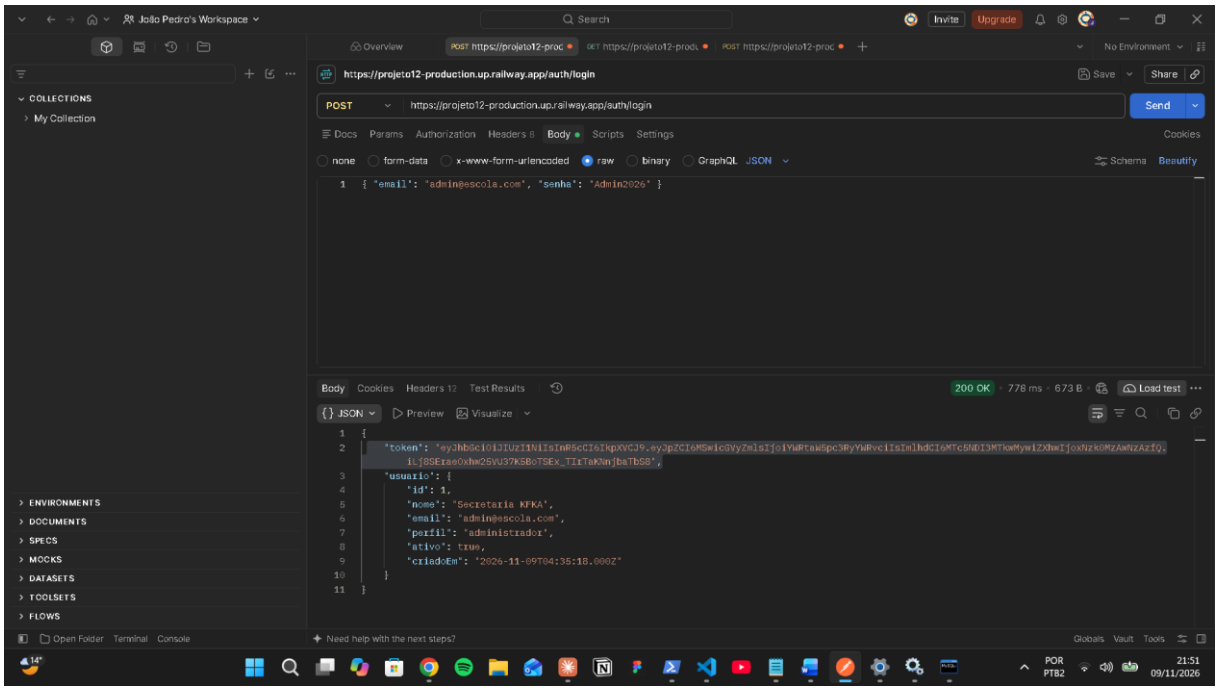
Dois middlewares protegem as rotas:

- verificarToken: exige o cabeçalho Authorization: Bearer <token> e confere a assinatura e a validade do JWT.
- somentePerfis(...): libera a rota só para os perfis indicados. Por exemplo, somentePerfis('administrador').

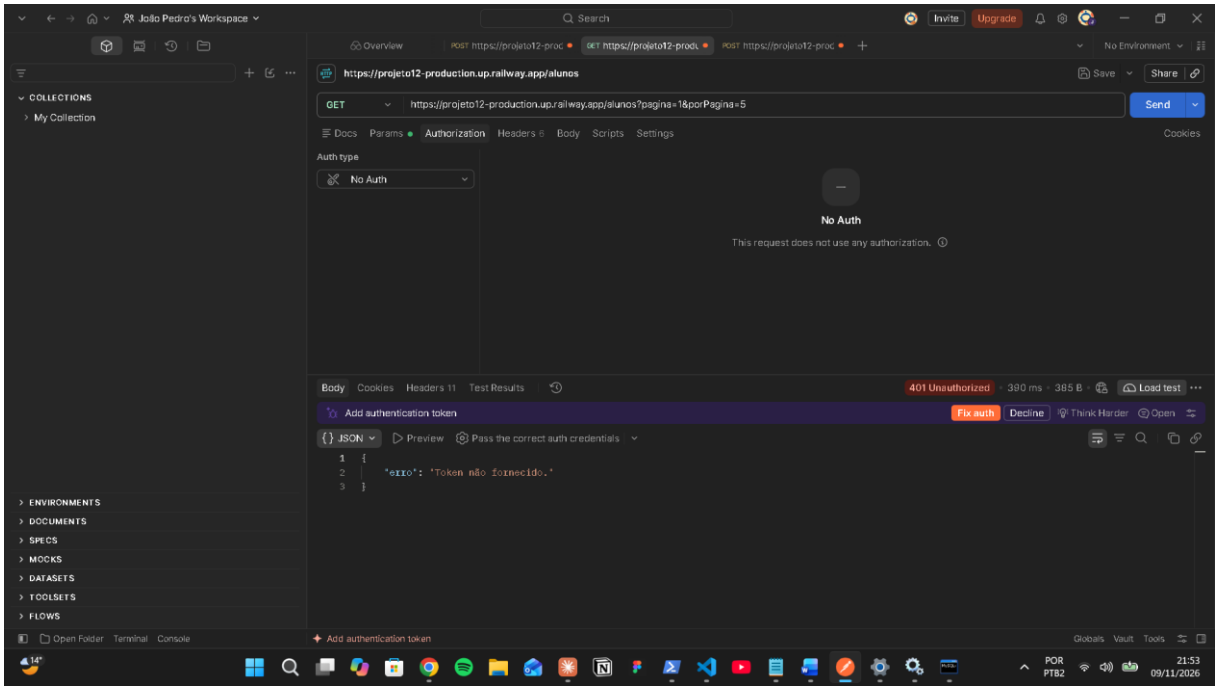
Além disso, o professor só acessa as turmas e disciplinas às quais está vinculado, e o responsável só vê os alunos vinculados a ele.

Na prática, uma requisição sem token recebe **401** com a mensagem "Token não fornecido.", e um token inválido ou vencido recebe **401** com "Token inválido ou expirado.". Um professor que tenta acessar GET /alunos recebe **403** com "Você não tem permissão para acessar este recurso.". Já um responsável que pede o relatório de um aluno que não é dele recebe **404** com "Aluno não encontrado entre os seus vinculados.", sem revelar se o aluno existe.

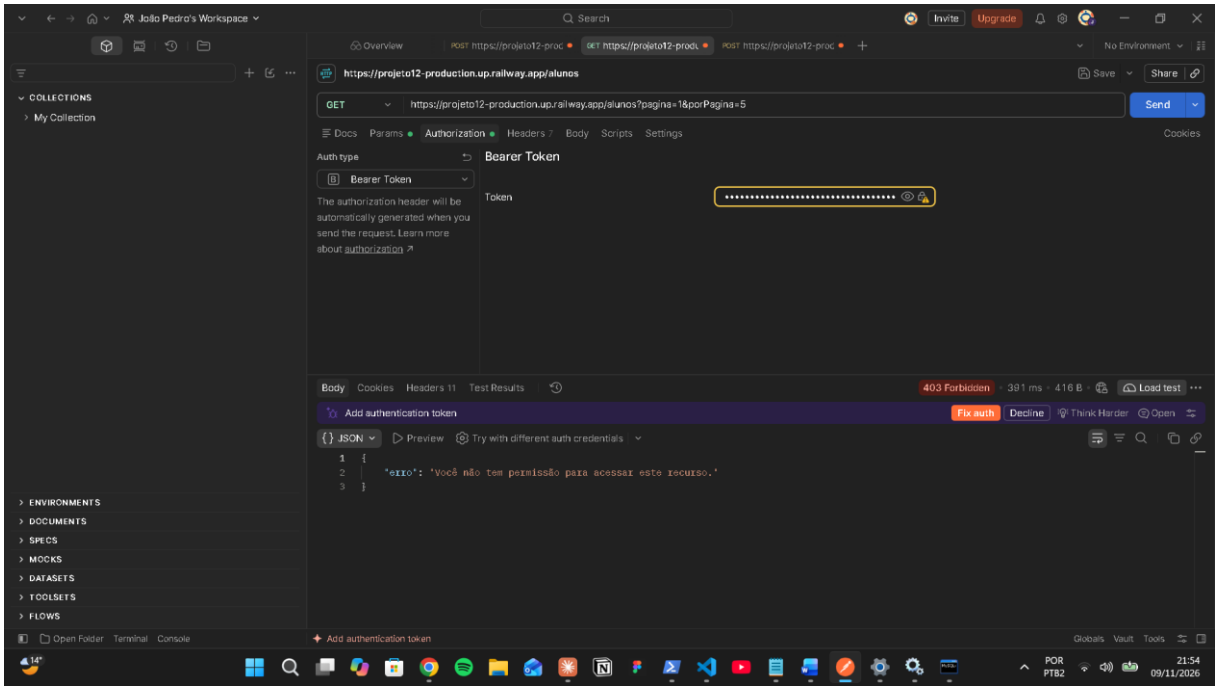
PRINT: Postman – POST /auth/login retornando 200 e o token



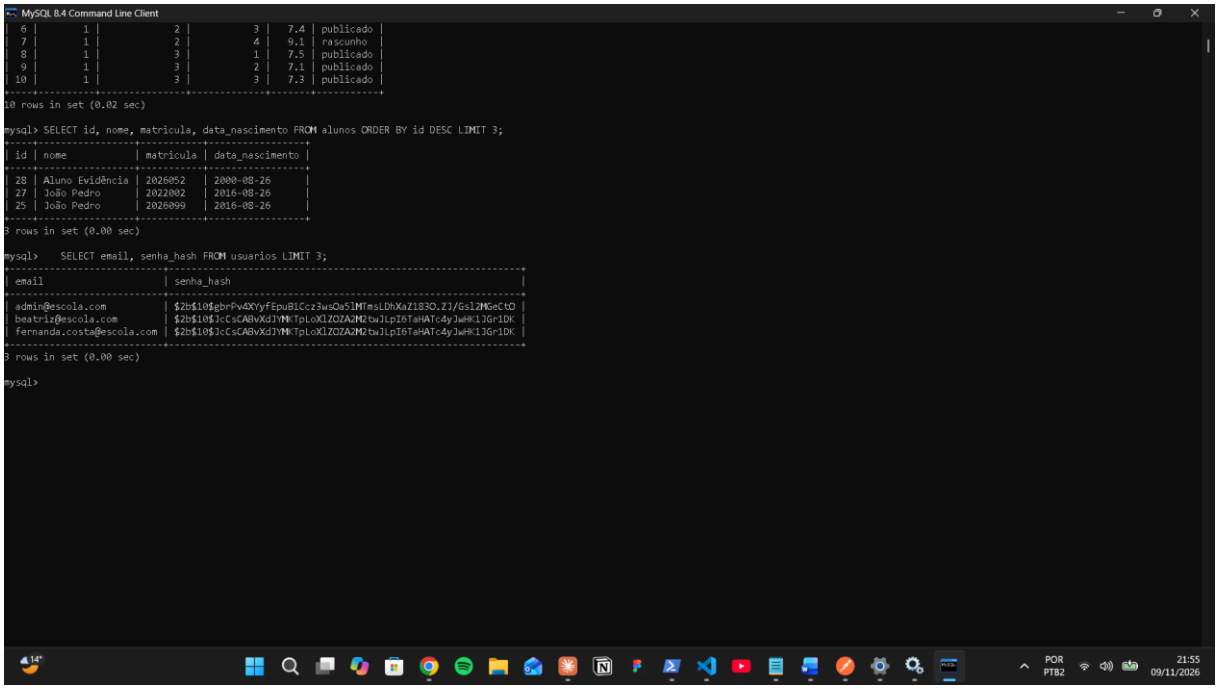
PRINT: Postman – GET /alunos sem token retornando 401
(aba Authorization em "No Auth")



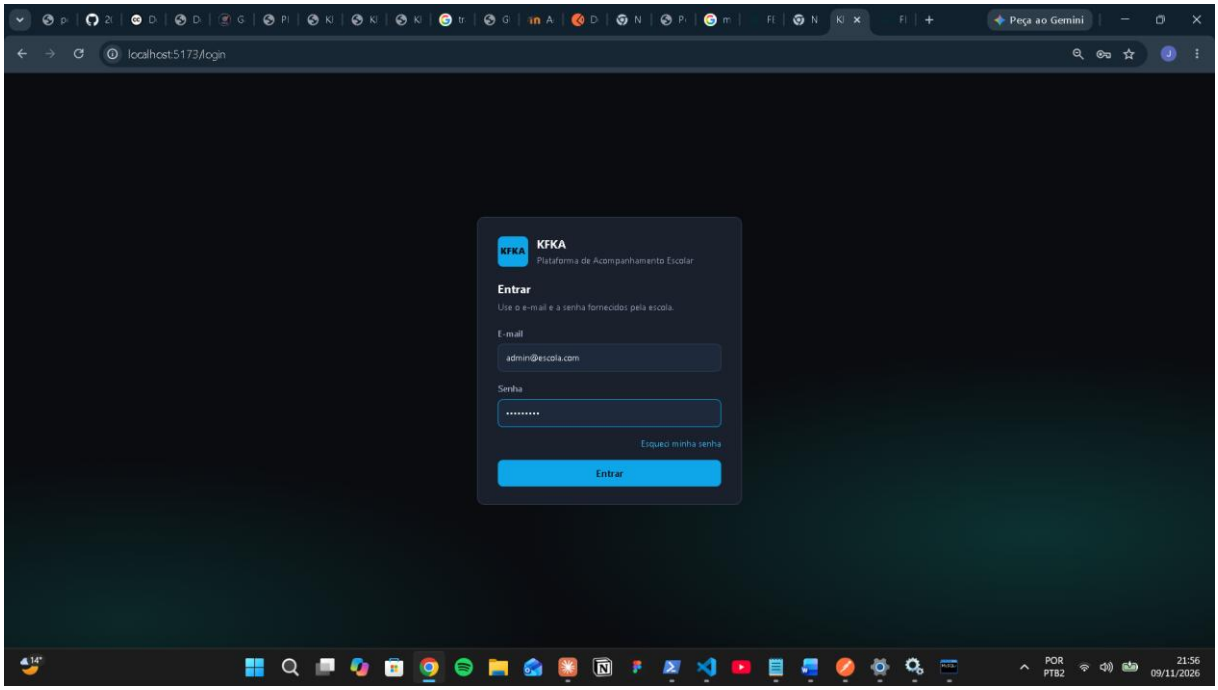
PRINT: Postman – GET /alunos com token de professor retornando 403]
(faça login com beatriz@escola.com e use esse token)



PRINT: Workbench – SELECT email, senha_hash FROM usuarios LIMIT 3]
(mostra que as senhas estão em hash bcrypt)



PRINT: Site – tela de login e a tela inicial após entrar como administrador]



4. Validação e erros padronizados

Toda entrada é validada no servidor, mesmo quando a interface já valida antes, porque a API pode ser chamada diretamente (como no Postman). Os erros seguem **um formato único**:

```
{ "erro": "Mensagem clara em português para quem usa o sistema." }
```

As regras de negócio lançam um ErroDeNegocio com o código HTTP adequado, e um tratador central converte esse erro na resposta. Erros inesperados viram 500 com uma mensagem genérica, sem expor detalhes internos.

4.1 Códigos usados

O código **400** indica dado inválido. É o caso de uma média fora da escala ("A média deve ser um número de 0 a 10, com no máximo uma casa decimal."), de um identificador que não é número ("Identificador inválido. Use um número inteiro positivo.") e de uma senha fraca, sem número ou com menos de 8 caracteres.

O código **401** indica que a pessoa não está autenticada ("Token não fornecido."). O **403** indica falta de permissão ou ação fora do prazo, como lançar notas com o bimestre fechado ("O período de digitação deste bimestre está fechado."). O **404** indica que o registro não foi encontrado ("Aluno não encontrado entre os seus vinculados.").

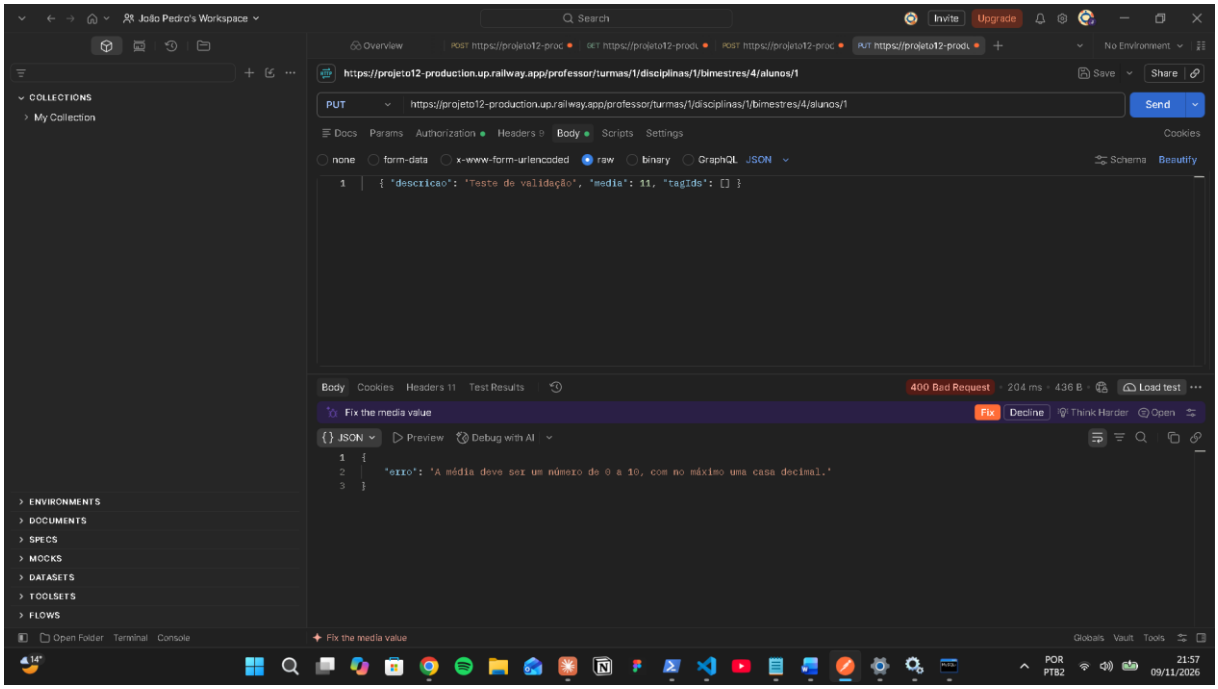
O código **409** indica conflito com algo que já existe ou com o fluxo do sistema. Ele aparece ao cadastrar uma matrícula repetida ("Já existe um aluno com essa matrícula."), ao matricular um aluno que já está em outra turma no mesmo ano, ao configurar um bimestre que se sobrepõe a outro ("O período se sobrepõe ao 4º

bimestre de 2026."), ao tentar publicar um acompanhamento que ainda é rascunho e ao registrar a ciência duas vezes ("A ciência deste bimestre já foi registrada.").

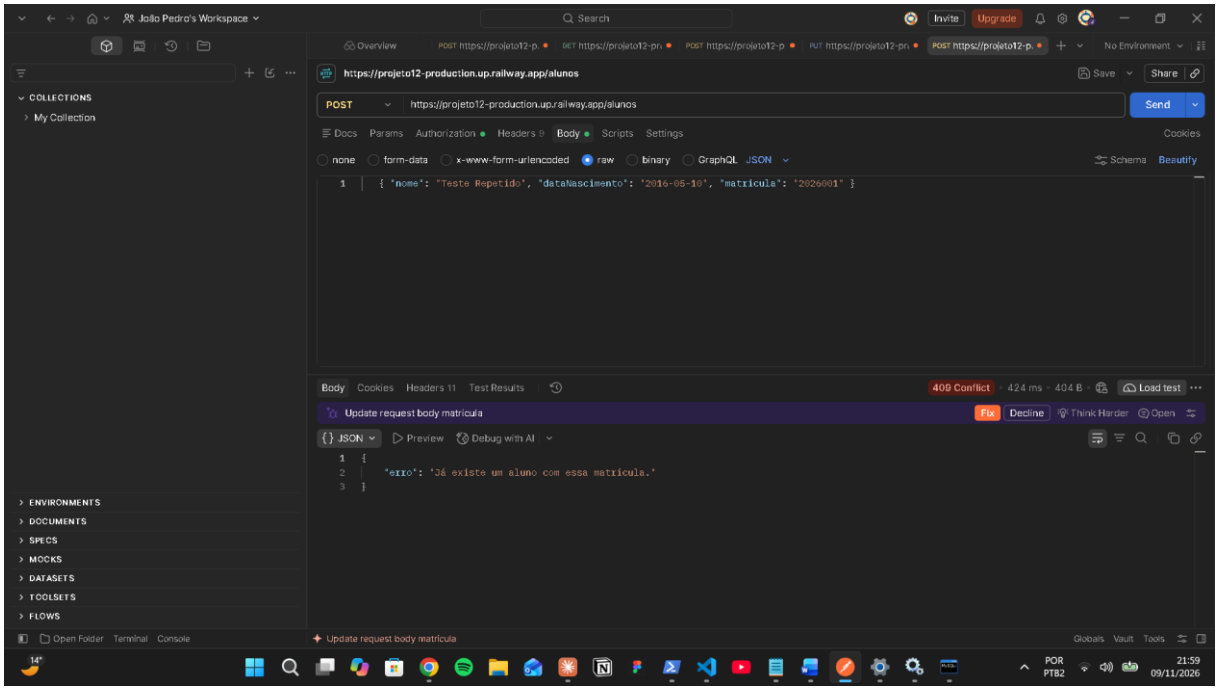
4.2 Validação também na interface

No site, os formulários usam campos obrigatórios e tipos adequados (e-mail, data, número) e mostram a mensagem de erro devolvida pela API logo abaixo do formulário, sem recarregar a página.

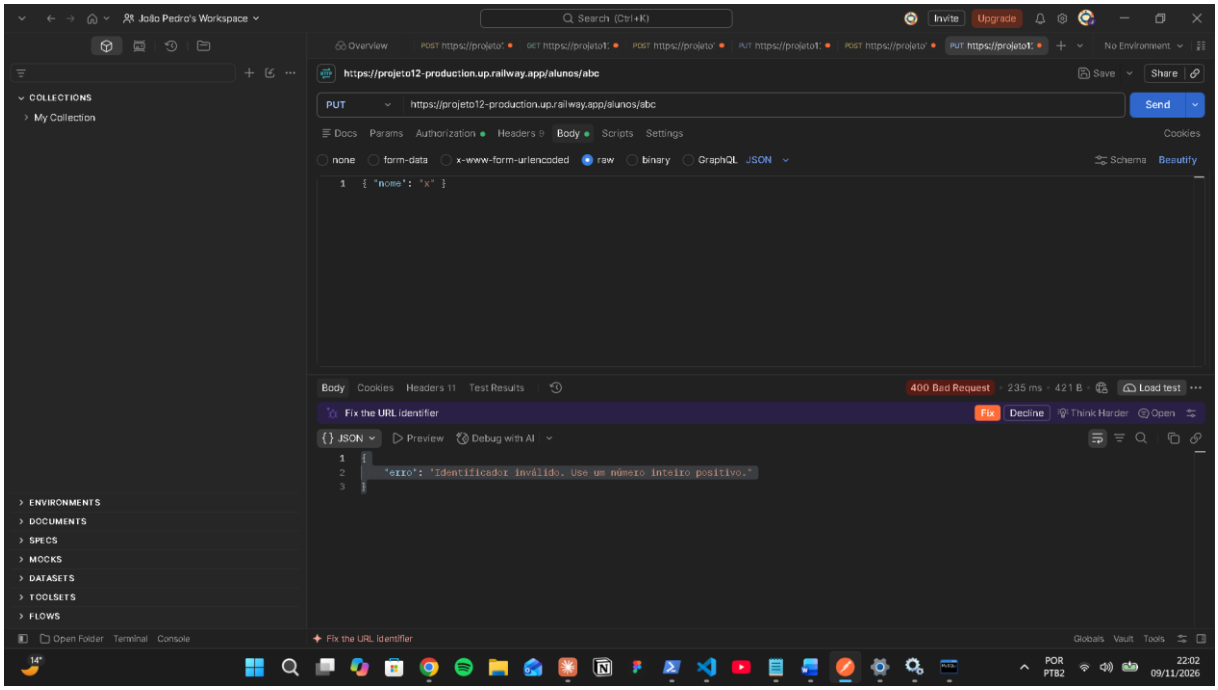
PRINT: Postman – PUT de lançamento com média 11 retornando 400
(corpo: { "media": 11, ... })



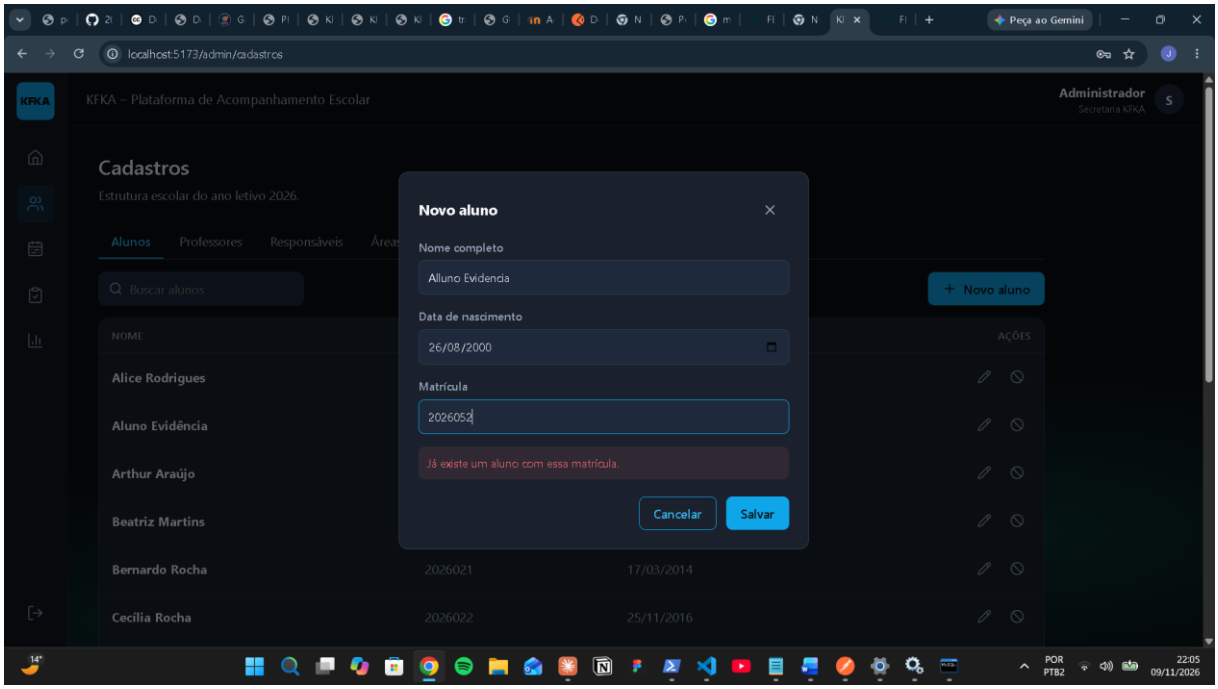
PRINT: Postman – POST /alunos com matrícula repetida retornando 409



PRINT: Postman – PUT /alunos/abc retornando 400 (identificador inválido)



PRINT: Site – mensagem de erro exibida em um formulário (ex.: matrícula repetida no cadastro de aluno)



5. Segredos fora do código

Nenhuma senha, chave ou endereço de banco está escrito no código. Esses valores vêm de **variáveis de ambiente**, lidas com process.env (e com o pacote dotenv no ambiente local).

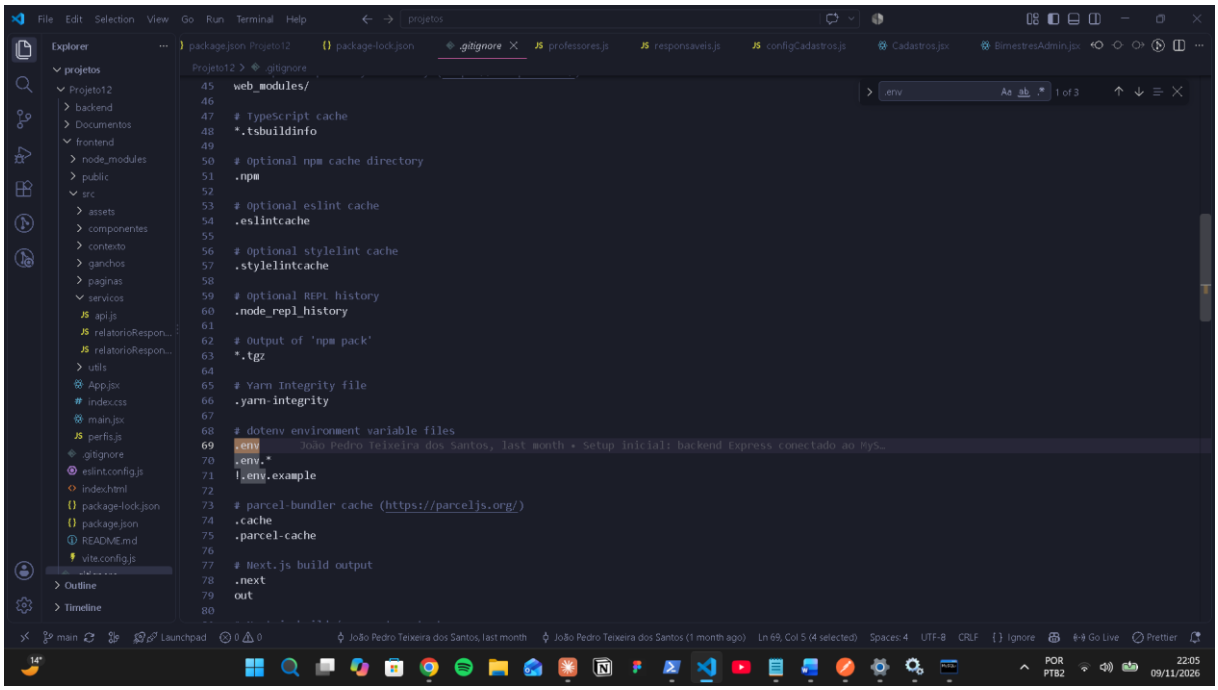
Na máquina local, os valores ficam no arquivo backend/.env, que está no .gitignore e nunca foi enviado ao GitHub. No repositório existe só o modelo backend/.env.example, com valores de exemplo, para quem for instalar o projeto. Em produção, os valores são cadastrados como variáveis do serviço no painel do Railway, e os dados de acesso ao MySQL são referências ao serviço do banco, sem digitar a senha.

O modelo traz as seguintes variáveis, com valores de exemplo:

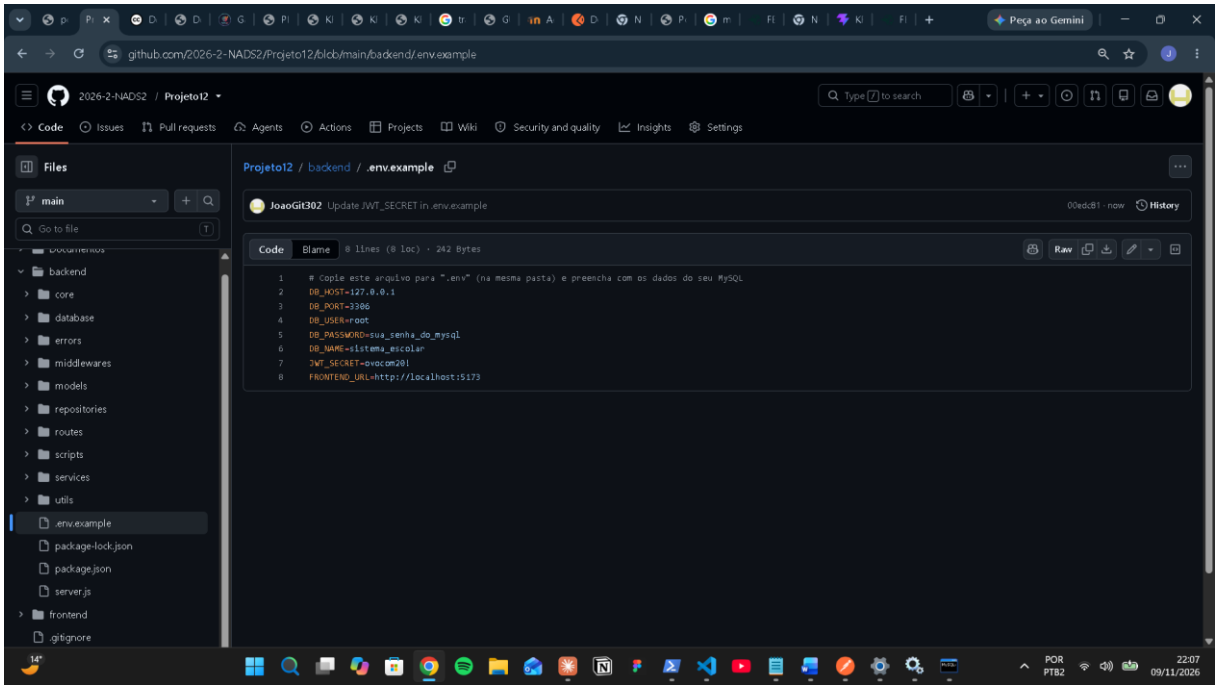
```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=sua_senha_do_mysql
DB_NAME=sistema_escolar
JWT_SECRET=troque_por_um_texto_longo_e_aleatorio
FRONTEND_URL=http://localhost:5173
```

O JWT_SECRET de produção é diferente do local, e o acesso público ao MySQL do Railway foi desativado depois da carga dos dados. Assim, o banco só é acessível pela rede interna do Railway.

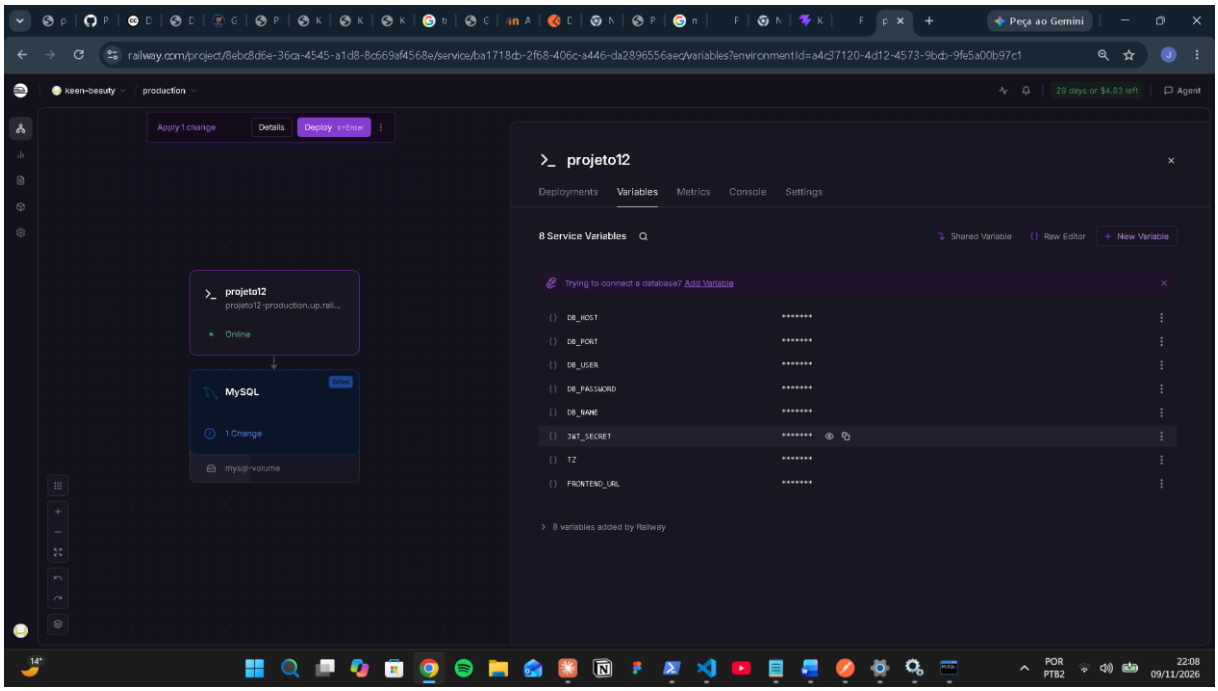
PRINT: VS Code ou GitHub – arquivo .gitignore mostrando a linha .env



PRINT: GitHub – arquivo backend/.env.example no repositório



PRINT: Railway – aba Variables do serviço projeto12 com os valores ocultos]
(sem as senhas nem o JWT_SECRET)



6. Aplicação publicada (front e back em produção)

O sistema está no ar em **https://projeto12-production.up.railway.app**. A publicação foi feita no **Railway** com dois serviços:

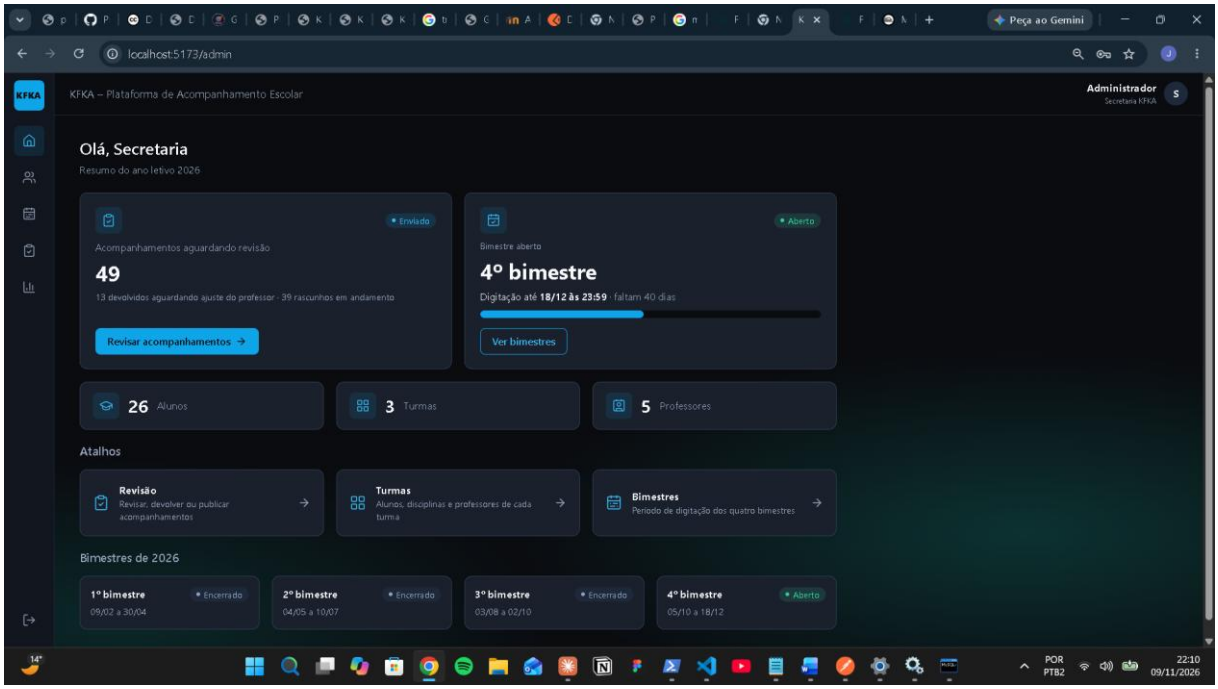
- **projeto12:** o servidor Node.js. Ele roda a API e também entrega a interface React já compilada (pasta frontend/dist). Assim, front e back ficam no mesmo endereço e não há problema de CORS.
- **MySQL:** o banco de dados, acessível só pela rede interna do Railway.

6.1 Como o deploy funciona

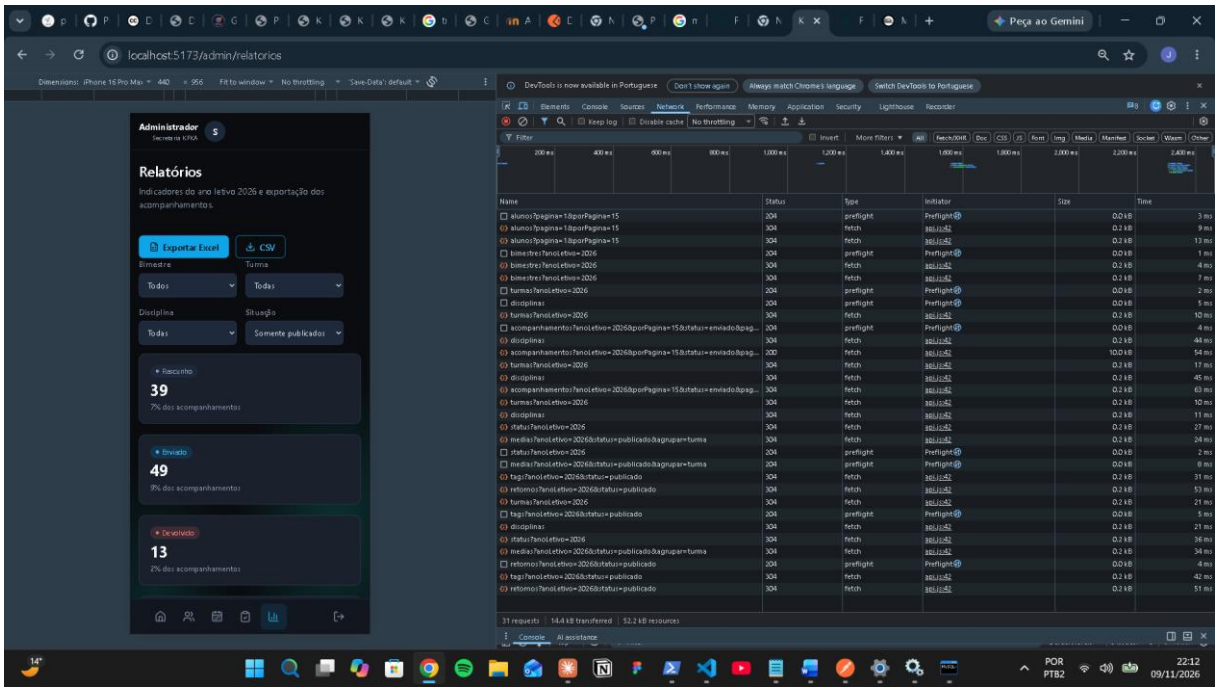
1. O package.json da raiz define o build: instala e compila o frontend (vite build) e instala as dependências do backend.
2. O comando de start sobe a API, que escuta na porta fornecida pelo Railway (process.env.PORT).
3. As rotas da API respondem em JSON. As demais páginas recebem o index.html do React, que cuida da navegação.
4. O fuso horário é fixado em America/São_Paulo, para que os prazos dos bimestres valham no horário de Brasília.

O site é **responsivo**: no celular, o menu vira uma barra inferior e as tabelas escondem colunas secundárias.

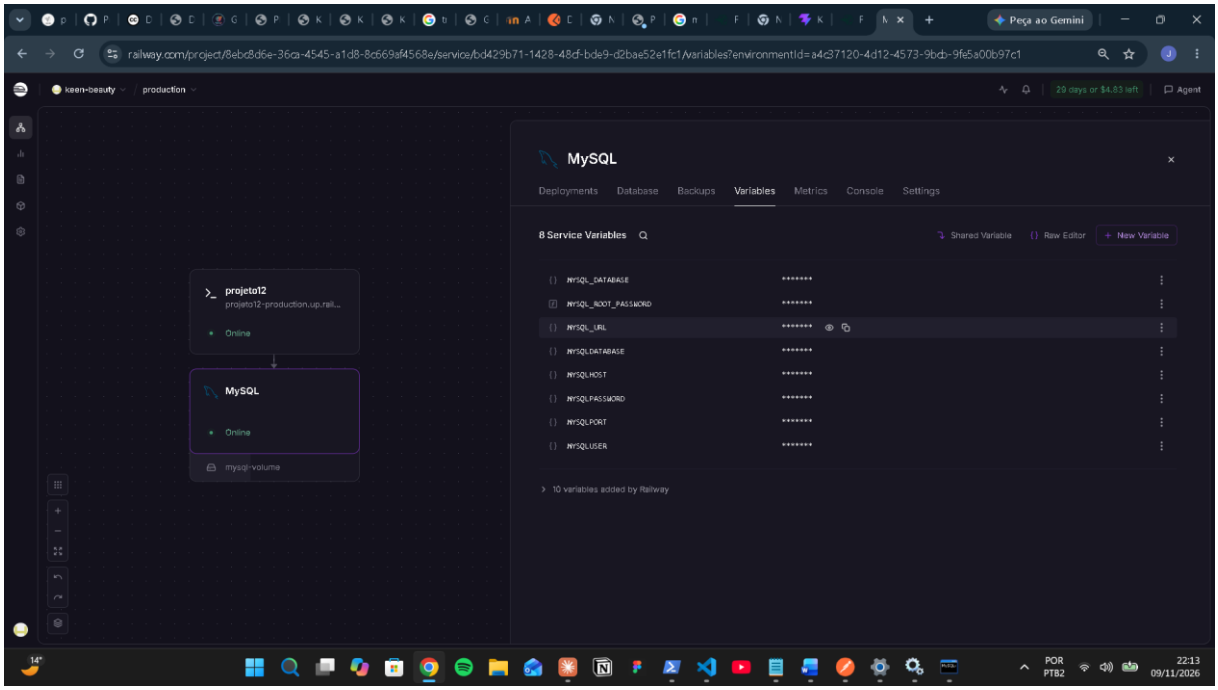
PRINT: Navegador – site publicado (URL visível) na tela inicial do administrador]



PRINT: Celular (ou DevTools em modo celular) – tela do responsável com o relatório]



[INSERIR PRINT: Railway – painel do projeto com os serviços projeto12 e MySQL ativos]


railway.com/project/8ebc8d6e-36ca-4545-a1d8-8c669af4568e/service/bd429b71-1428-48df-bde9-d2bae52e1fc1/variables?environmentId=a4c37120-4d12-4573-9bdb-9fe5a00b97c1
keen-beauty
production
MySQL
Deployments
Database
Backups
Variables
Metrics
Console
Settings
8 Service Variables
Shared Variable
Raw Editor
New Variable
MYSQL_DATABASE
MYSQL_ROOT_PASSWORD
MYSQL_URL
MYSQLDATABASE
MYSQLHOST
MYSQLPASSWORD
MYSQLPORT
MYSQLUSER
10 variables added by Railway
projeto12
projeto12-production.up.rail...
Online
MySQL
Online
mysql-volume

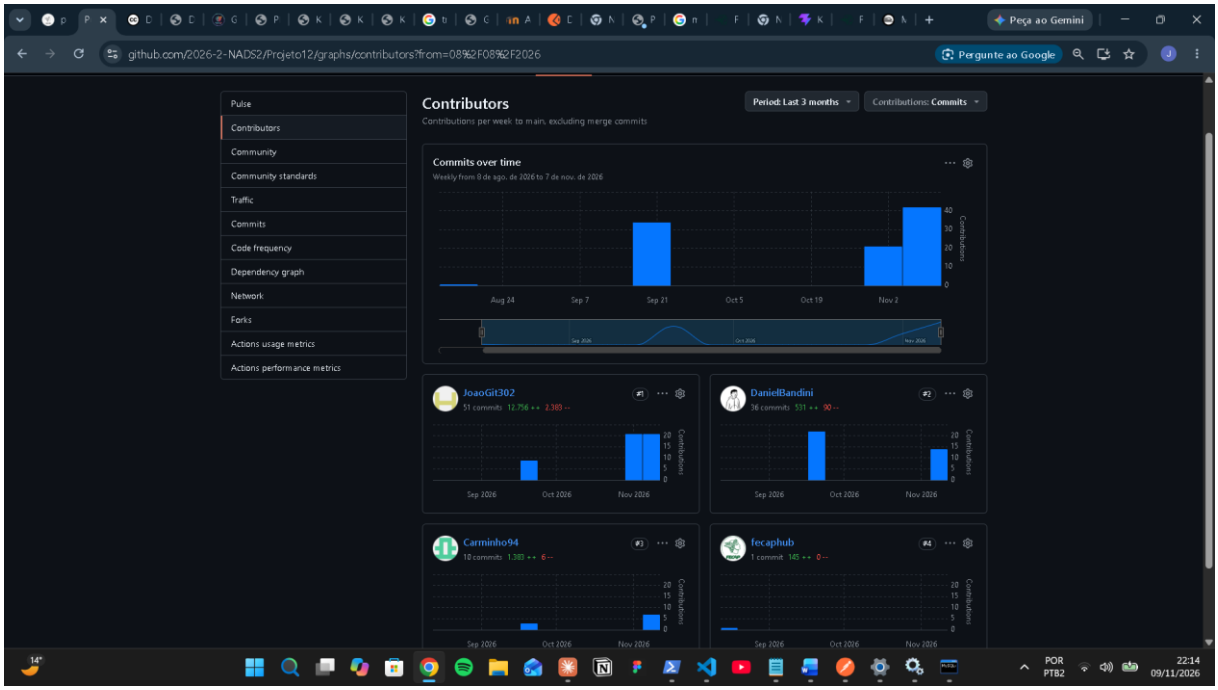
7. Repositório com commits de todos os integrantes

O código está em **https://github.com/2026-2-NADS2/Projeto12**. O histórico tem mais de 80 commits, feitos ao longo do semestre pelos integrantes do grupo.

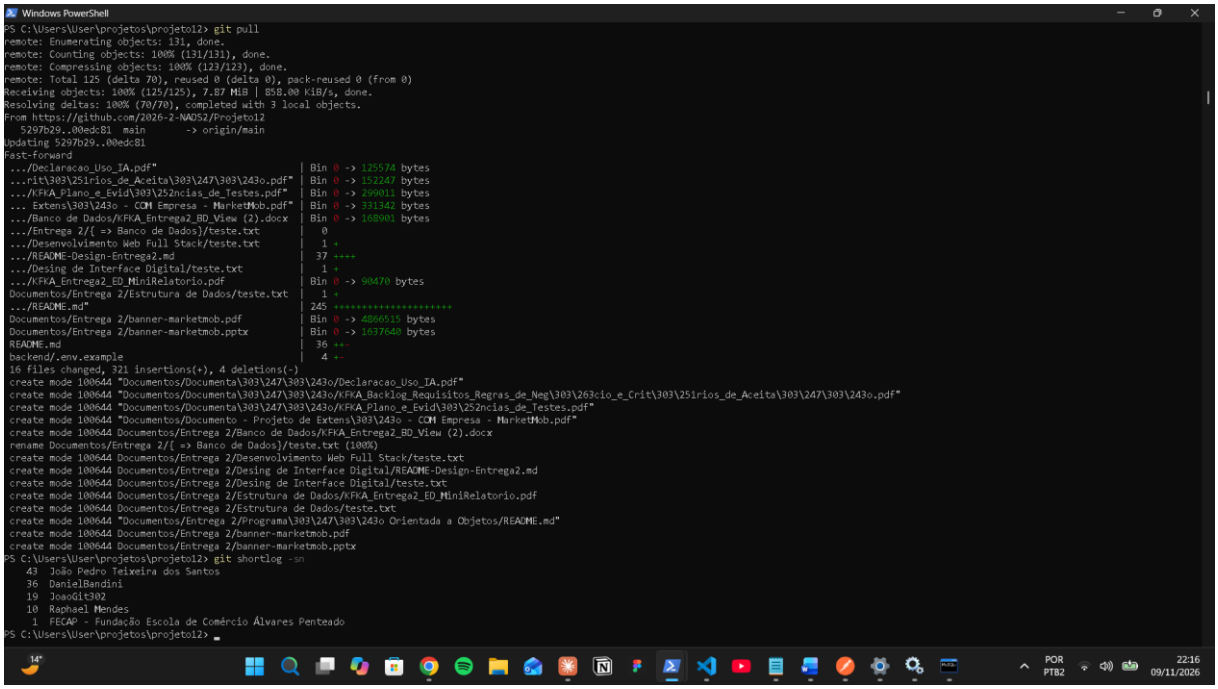
Pela contagem do comando git shortlog -sn na data desta entrega, **João Pedro Teixeira dos Santos** tem 61 commits, distribuídos em três identidades (joaopedroteixeira302, JoaoGit302 e 26028540@fecap.br), porque ele trabalhou em máquinas diferentes, na faculdade e no notebook. **Daniel Bandini Grande** tem 24 commits, pela conta DanielBandini, e **Raphael Carmo Mendes** tem 3 commits, pela conta Raphael Mendes.

O repositório está organizado em cinco partes. A pasta backend/ tem a API Node.js com Express, com o schema.sql em backend/database/. A pasta frontend/ tem a interface React com Vite. A pasta Documentos/ reúne as entregas por disciplina, o backlog, o plano de testes e a declaração de uso de IA. O package.json da raiz define o build e o start usados no deploy, e o README.md explica a instalação, os acessos de demonstração e os requisitos atendidos.

PRINT: GitHub – página Insights > Contributors (ou a lista de commits com os autores)



PRINT: Terminal – resultado de git shortlog -sn



Resumo do atendimento

Os sete itens pedidos foram atendidos. A API com as rotas principais está descrita na seção 1 (Figuras 1 e 2), e a persistência em banco na seção 2 (Figuras 3 a 5). A autenticação e as rotas protegidas estão na seção 3 (Figuras 6 a 10), e a validação com erros padronizados na seção 4 (Figuras 11 a 14). Os segredos fora do código aparecem na seção 5 (Figuras 15 a 17), a aplicação publicada na seção 6 (Figuras 18 a 20) e os commits de todos os integrantes na seção 7 (Figuras 21 e 22).

Documentos complementares estão no repositório, na pasta Documentos/Documentação: Backlog, Plano e Evidências de Testes e Declaração de Uso de IA.